CompSci 390-03-26S

Distributed and Networked Systems

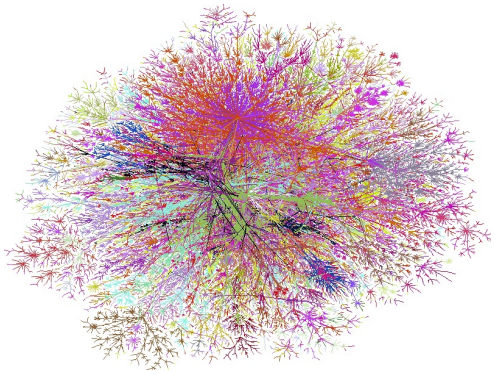
Cryptosystems and TLS

Path security

Anyone who owns a network on the transmit path can:

- intercept the packet
- transform/corrupt the bits
- block the packet or misdirect

"Internet treats censorship as failure and routes around it."



Security properties

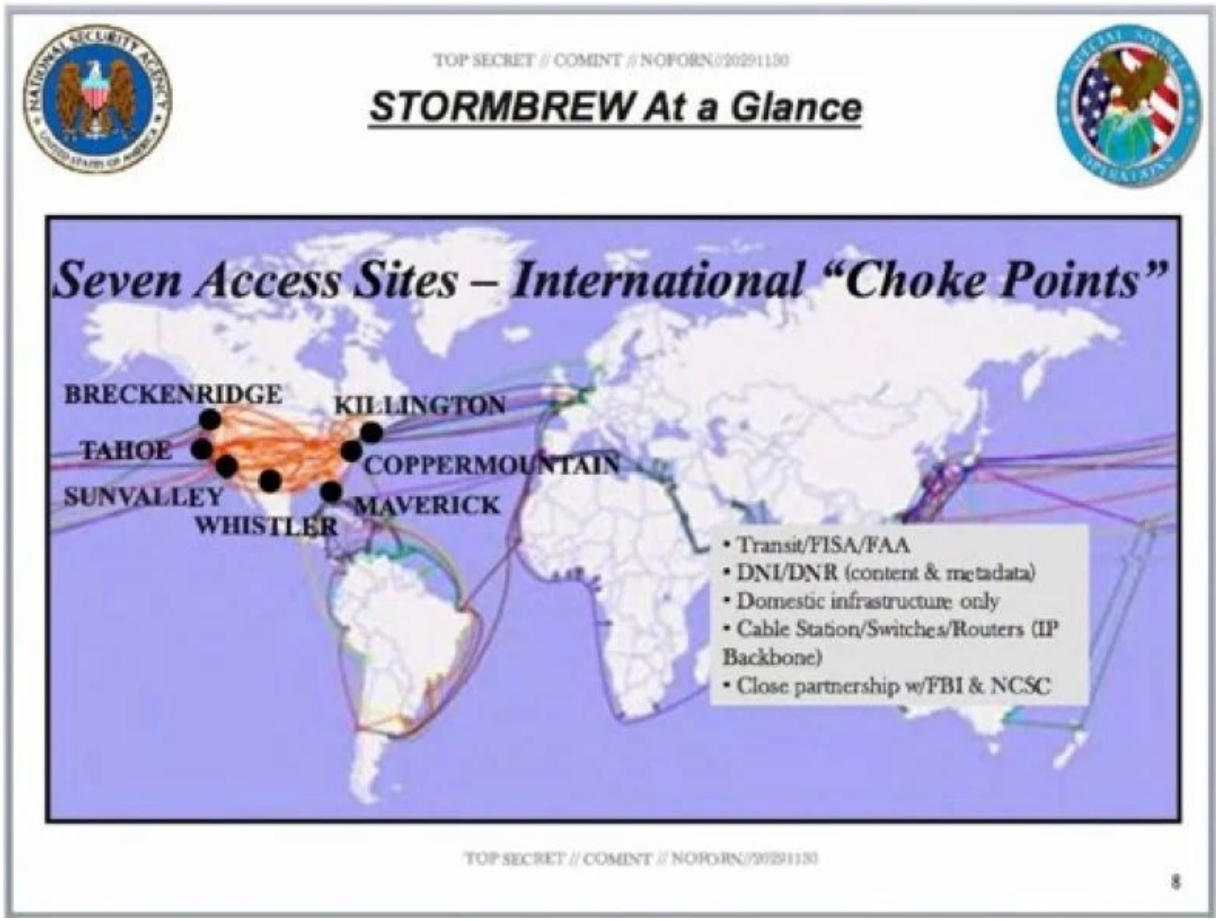
How to provide secure communication between Alice, Bob, Cindy...and Mal?

- **Authentication**: who sent the message?
- **Integrity**: is it what they intended to send?
- **Confidentiality**/privacy: no eavesdropping

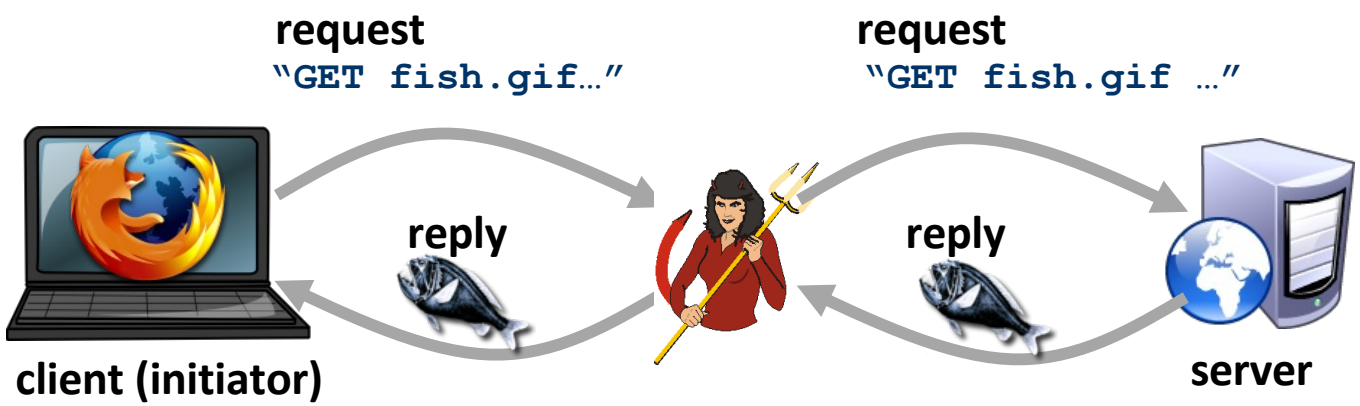
Two cases to consider:

- **Transport Layer Security** (TLS): encrypted connections
- **Secure messaging**, e.g., Signal

Stellar Wind and Stormbrew



Mal-in-the-Middle attack



If an attacker can interpose between two interacting endpoints, she can read and modify the messages at will without detection by either victim. We need some crypto.

Classically a Man-in-the-Middle attack (MITM).

The Adversary (Mallory, Eve)

- **Who is the Adversary?** What is their objective?
  - Immaterial! Philosophy: privacy is a right, and crypto is a public good.
  - In practice, views differ. Crypto is a focal point of policy tussle.
- **Threat model**. What powers does the adversary have?
  - Observe the packet flow, inject or modify packets.
  - Save copies to crack later
  - Compromise an endpoint and steal keys

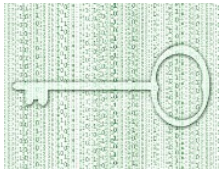
- **Assumption**: adversary is unable to control an endpoint during a session or steal clear copies of decrypted data.
- More precisely, such attacks are outside the threat model.

Our approach to cryptosystems

- Crypto is based on various mathematics that seem to allow **one-way** functions and **trapdoor** functions as building blocks.
  - Even knowing public function f, and knowing f(x), it is **"computationally infeasible"** to determine x or to discover y with f(y)=f(x).
  - Preferred mathematical basis has changed over time, e.g.,: RSA→elliptical→lattice (quantum-resistant)
- Crypto building blocks are combined in **constructions** that offer specific properties or defend against various attacks.
- We focus on the constructions and their properties, from the bottom up, glossing over the mathematics.

Four crypto primitives

Secure hashing
useful for
fingerprinting data

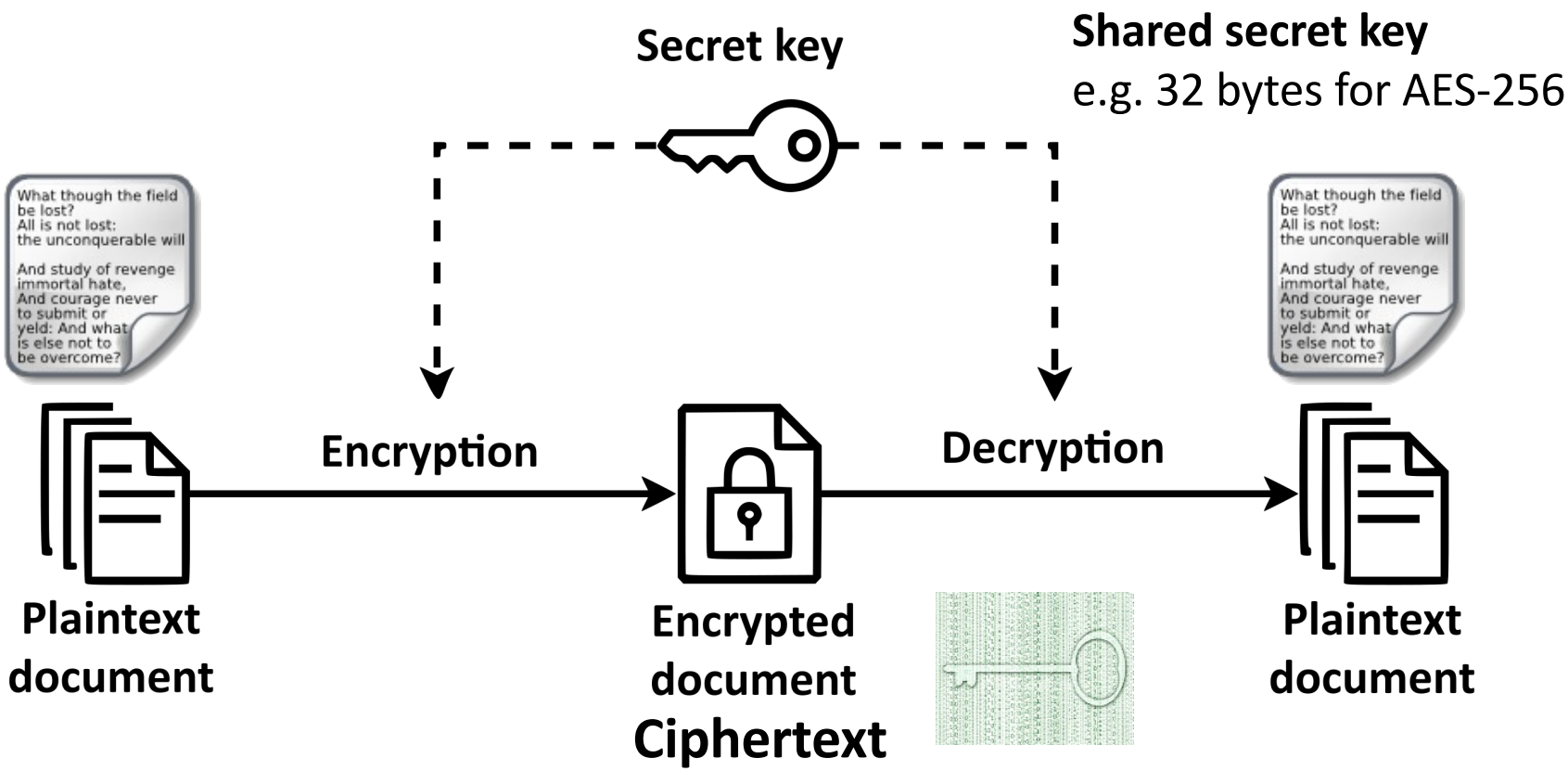


Encrypting /Decrypting

Signing / Authenticating

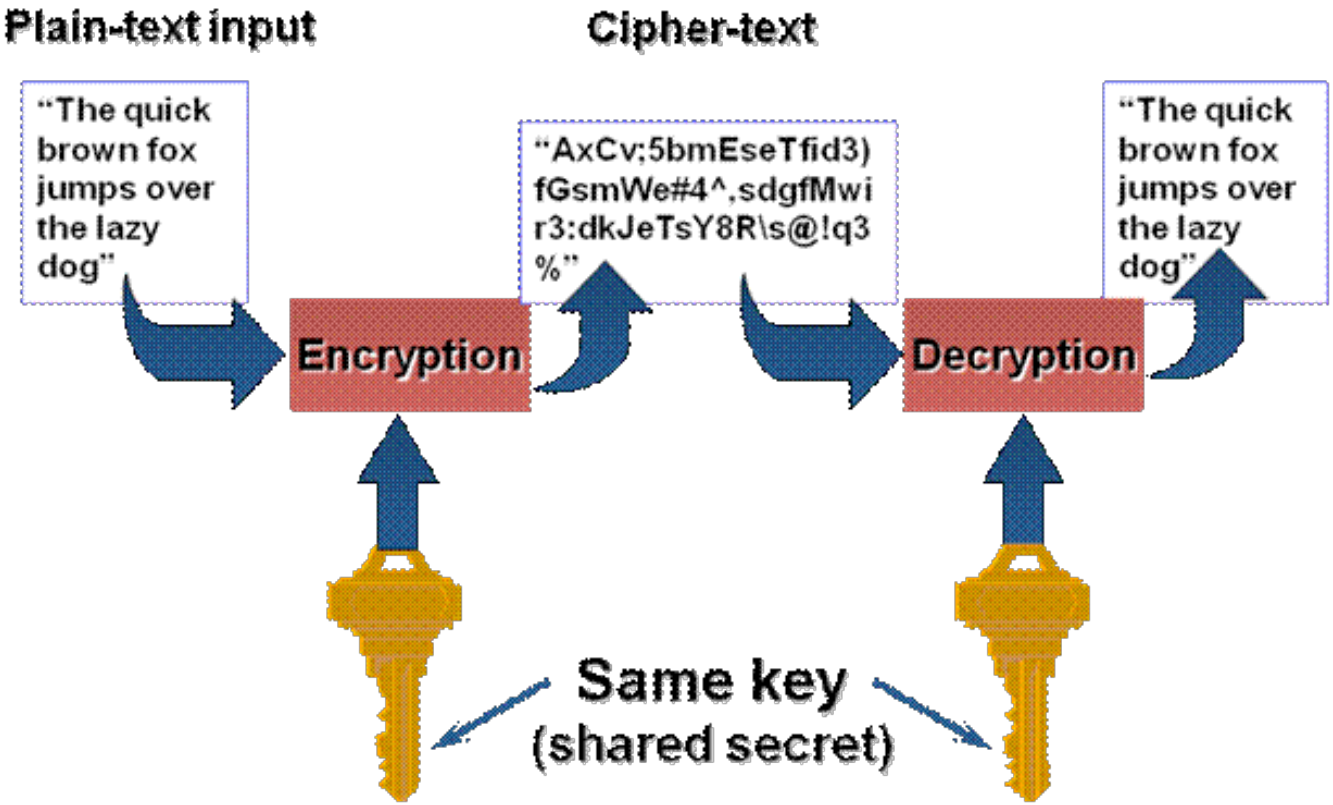
Key exchange / agreement

Symmetric crypto



https://commons.wikimedia.org/wiki/File:Simple_symmetric_encryption-en.svg

Symmetric crypto

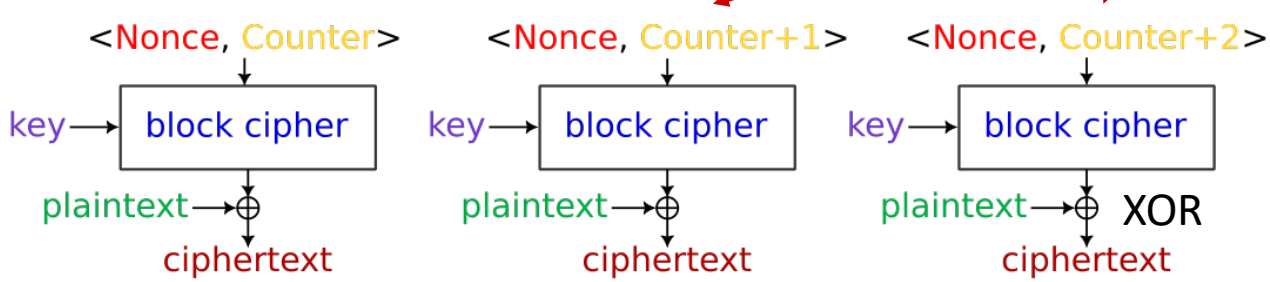


Borrowed from https://spideroak.com/blog/20130523083520-drink-your-ovaltine-intro-to-encryption-101

AES: Standard for symmetric crypto

- Advanced Encryption Standard
- Various key sizes and modes for streaming 128-bit blocks
- Fast and cheap, parallelizable
- Hardware acceleration available
- Lots of shifts, rotation, and XOR

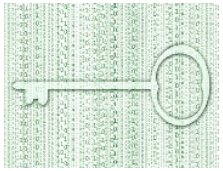
Same nonce ("number used once"), counter increments.



Using symmetric crypto (AES)

Suppose Alice and Bob have a shared secret to start with:

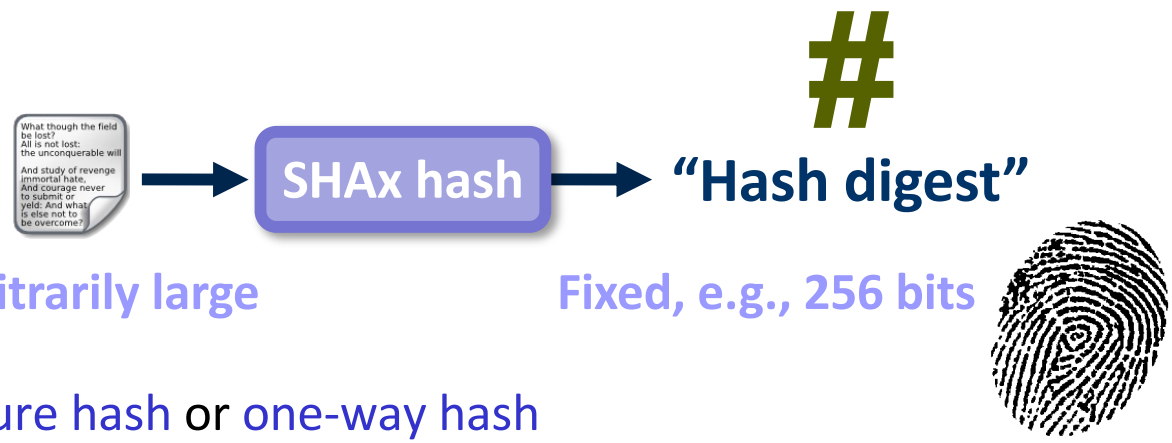
- "Keying material": a pattern of bits that looks "random".
- They can use it to derive a **session key** for AES.
- They can use the session key to encrypt their transfers.
- That assures **confidentiality**: Mal cannot eavesdrop.



But to build **transport layer security (TLS)** we still need:

- **Integrity**. How to detect if an adversary tampers with the bits? (Next, but we need a few more building blocks.)
- **Key agreement**. How to establish a shared secret? (Later)

Cryptographic hashes



Also called a secure hash or one-way hash

- The hash function h takes a byte sequence M as input.
  - E.g., ~~SHA0, MD5~~, ~~SHA1~~, SHA2/**SHA-256**, SHA3, MD6
  - SHA-x: Secure Hashing Algorithm
- Very efficient, produces a small fixed-width result: h(M)
- Result is called a hash, checksum, fingerprint, digest

Properties of Secure Hashing (SHA)

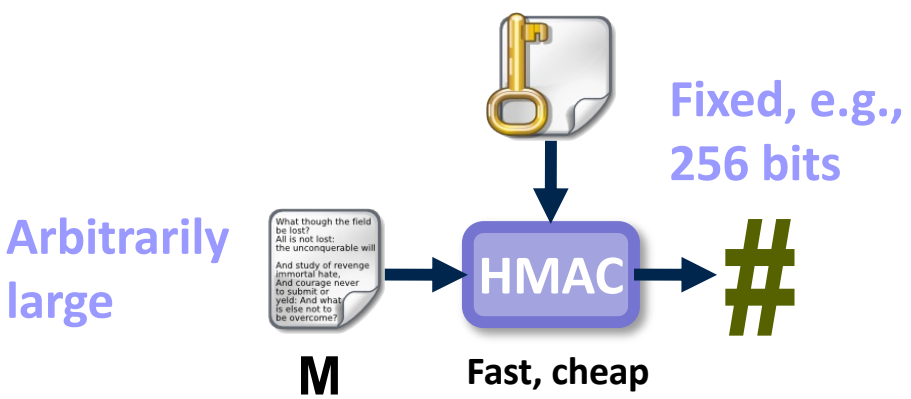
- Collision-resistant
  - Sure, there exist distinct M_1 and M_2 such that $h(M_1) == h(M_2)$.
  - But such **collisions** are vanishingly rare and "very hard" to find.
- One way
  - Given digest, cannot generate an M with h(M) == digest.
  - Such collisions are "very hard" to find.
- Secure
  - The digest does not help to discover any part of M.

Cheap

"Computationally infeasible"

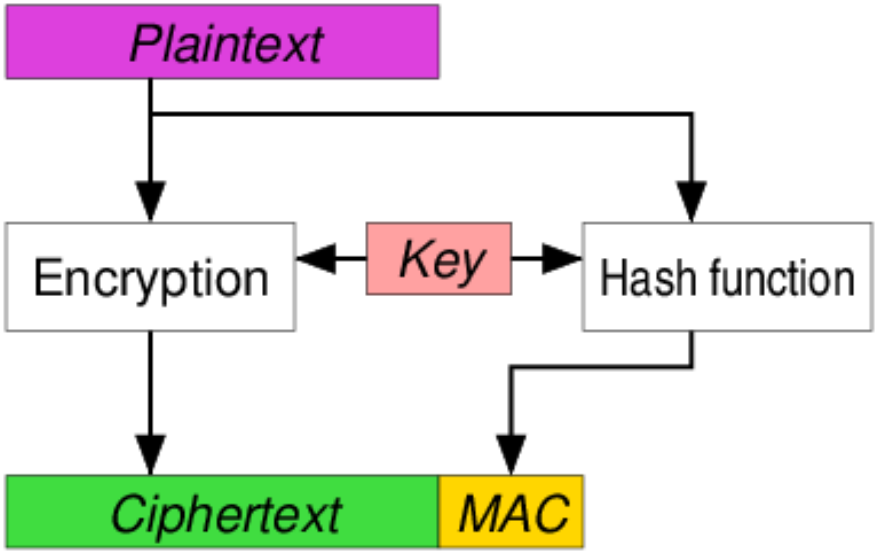
HMAC: Hashed Message Authentication Code

- HMAC is a SHA parameterized with a key: a keyed hash.
- Why "MAC"? Send a message M, append HMAC(k, M). If receiver knows k, then it can authenticate M:
  - Hash matches? →M is untampered from a sender that knows k.
  - Adversary must know k to tamper the message without detection.
- Other uses! (Later: HKDF)



Authenticated Encryption (AE)

- Construction of AES+SHA/HMAC
- Encryption → **confidentiality**
- MAC → **integrity**
- Some variants encrypt before hashing, and/or use two keys.
- **AEAD**: AE with Associated Data: the AD is hashed into MAC for integrity, but not encrypted.
- In TLS the packet headers are AD: unencrypted but covered by MAC.



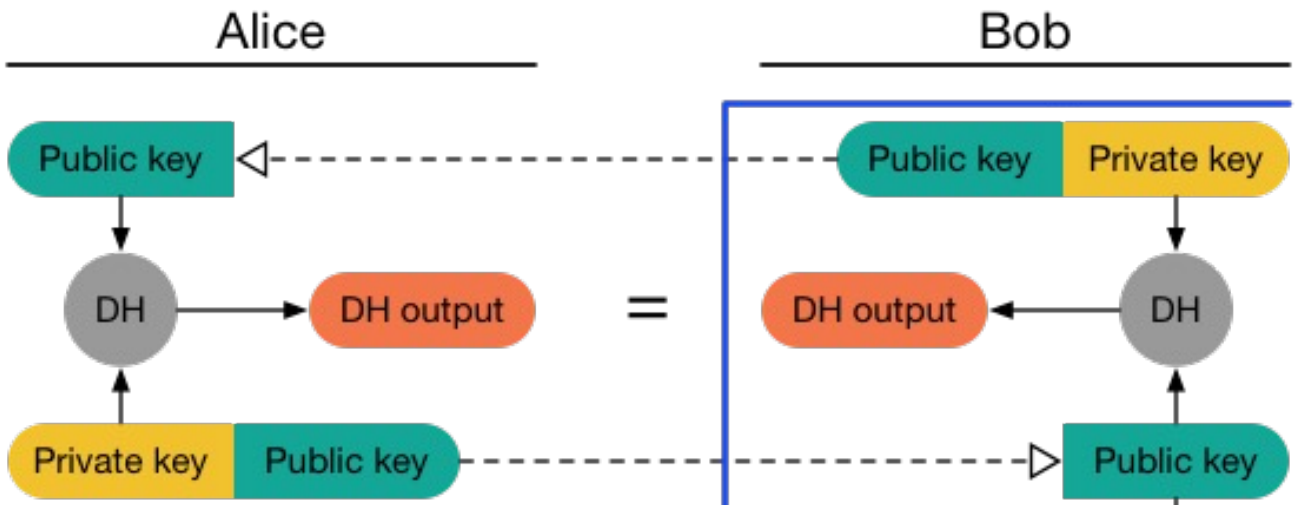
TLS today uses AES for encryption and SHA-2 for hashing, with various qualifying key sizes.

How to share a secret?

- OK, so: if Alice and Bob have a shared secret, they can communicate securely (with confidentiality, integrity).
  - Derive session key k from the shared secret
  - Use k with AES and/or AEAD
  - Works....if adversary does not learn k!
- But how can Alice and Bob agree on k and keep it secret?
- This is a, uh, key problem, called **key exchange**, **key agreement**, or **key distribution**.

Diffie-Hellman key exchange

- **Public key cryptography**: each participant creates their own **keypair** with a public key and a private key. And never reveal the private key to anyone!
- They exchange their public keys over an open channel.
- DH(pubA, pubB, privA) == DH(pubA, pubB, privB) → shared secret

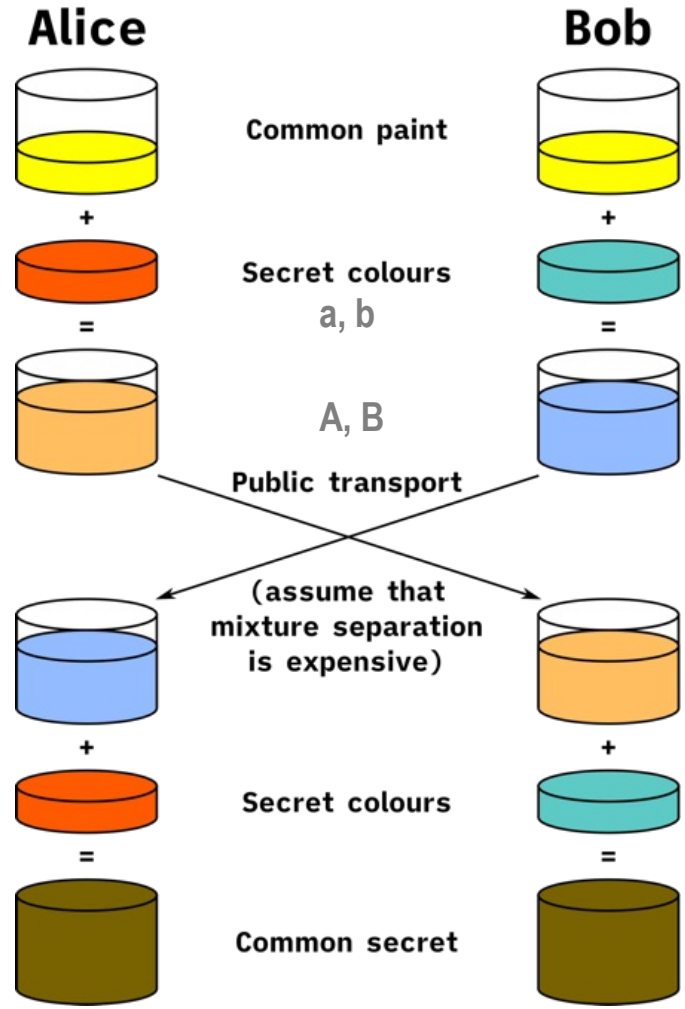


https://signal.org/docs/specifications/doubleratchet/

Diffie-Hellman classic

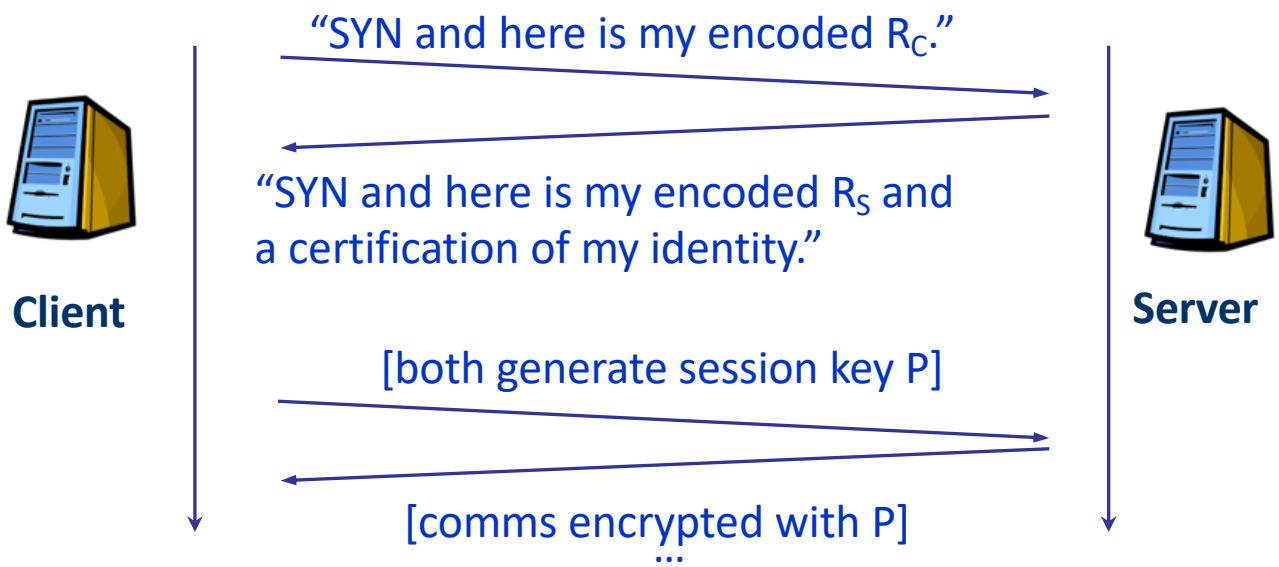
- Choose base g and prime p.
- **Everyone** knows g, p.
- Choose "secret colors" R_x.
  - $a = R_{Alice}$, $b = R_{Bob}$
- Share $A = g^a \bmod p$.
- Share $B = g^b \bmod p$.
- Key = $B^a \bmod p$
- Key = $A^b \bmod p$. Same key!
- **Discrete logarithms**
- **Today:** elliptical curve **ECDH**

https://en.wikipedia.org/wiki/Diffie—Hellman_key_exchange
Public domain image



Transport Layer Security 1.3

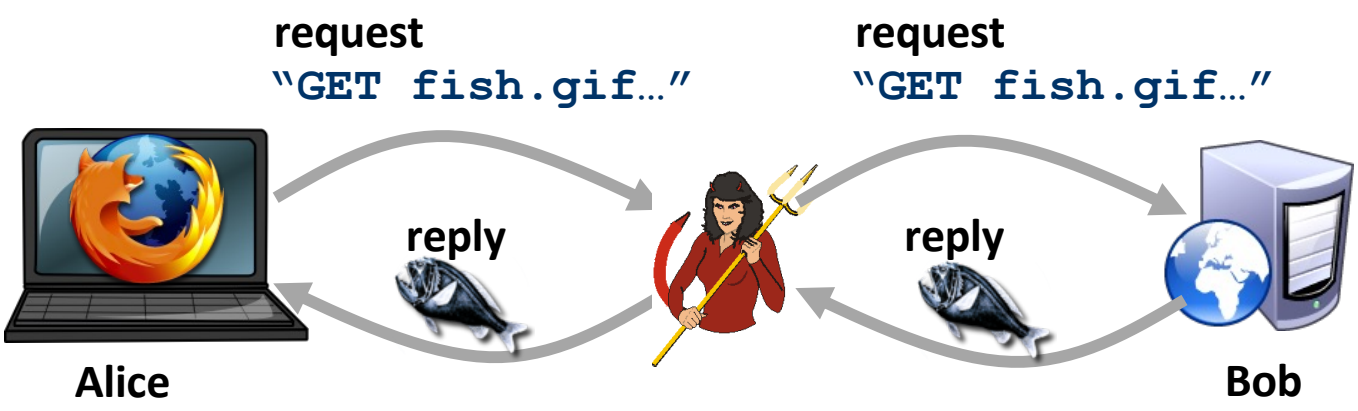
Cartoonishly oversimplified



Client and server generate secret (private) random numbers R_C and R_S, and share (public) encodings of their secret Rs. Diffie-Hellman enables each to generate same shared session key P from its R and peer's encoded R.

Are we there yet?

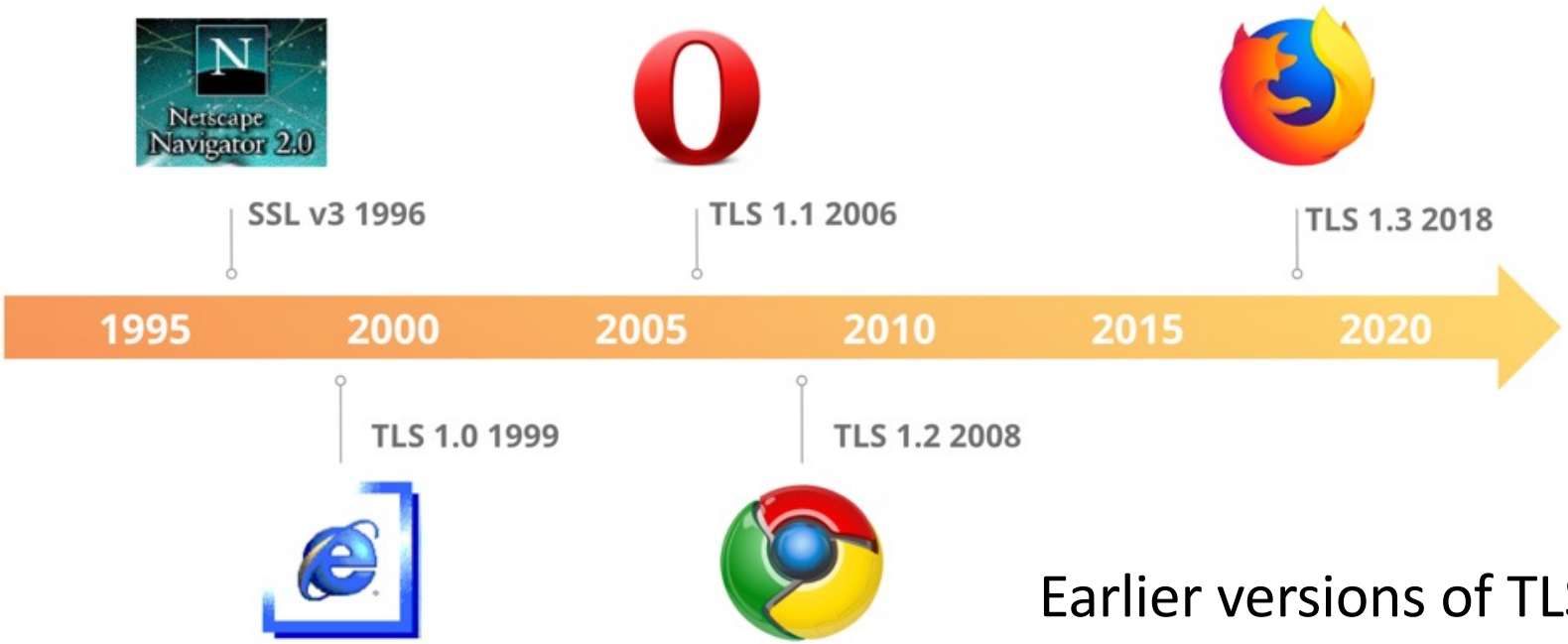
- OK, so we know how any parties on the network can establish a secure channel: confidential with integrity.
- But how does Alice know she is talking to Bob? And not Mal?
- **Authentication** with certificates and digital signatures (later).



Diffie Hellman: spelling it out

- Both endpoints generate private secrets.
- Each endpoint uses its secret to derive a public value.
- The endpoints exchange their public values.
- Each endpoint generates a session key from its secret and its peer's public value
- An eavesdropper may know both public values, but not either of the secrets. It cannot compute the session key.
- One-way math based on discrete logarithms or elliptic curves.

TLS timeline

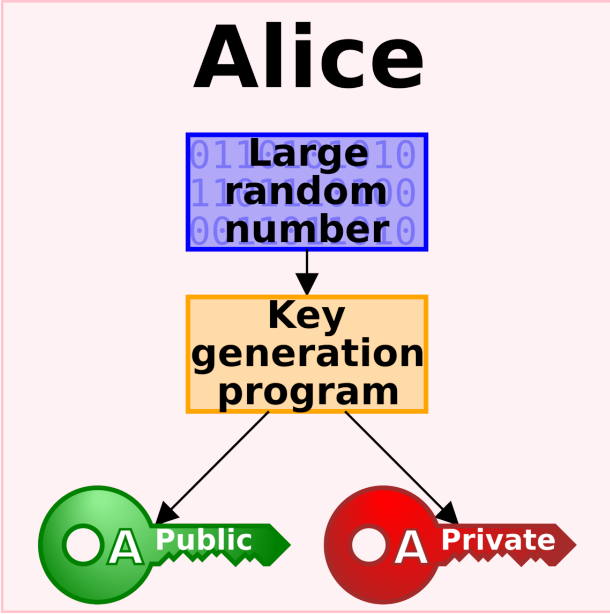


Earlier versions of TLS used RSA public-key crypto

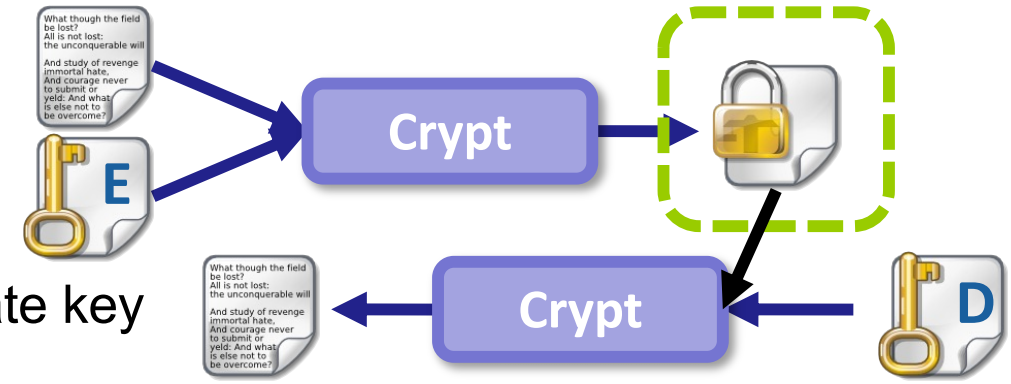
https://blog.cloudflare.com/rfc-8446-aka-tls-1-3/

RSA asymmetric crypto

- Rivest Shamir Adleson 1977
- Each subject/principal possesses a keypair.
- Keep one key private; the other is public.
- **Either key** can be used to encrypt; the **other** key is used to decrypt.
- Decrypt(K, Encrypt(K^{-1}, M)) = M



RSA works both ways



A's private key
or
A's public key

A's public key
or
A's private key

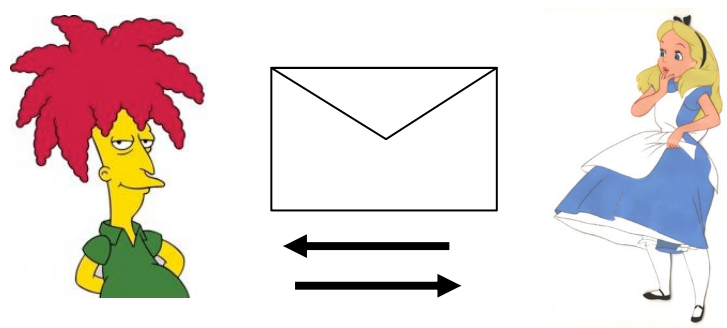
[Landon Cox]

How to use RSA?

Consider:

- A sends a message to B, encrypted with A's **private** key.
- B sends a message to A, encrypted with A's **public** key.

Benefits? Other possibilities?

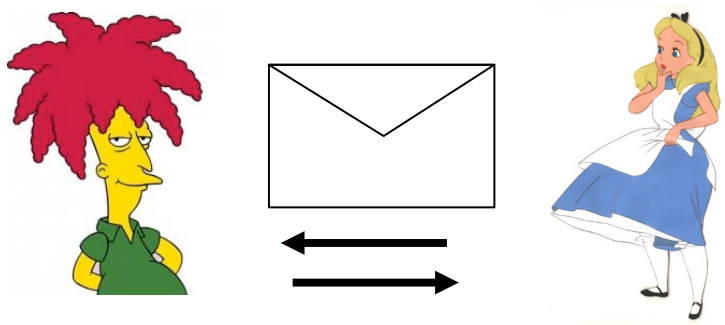


How to use RSA?

If B knows A's public key, that is sufficient for:

- A→B messages: authenticated but not secret.
- B→A messages: secret but not authenticated.

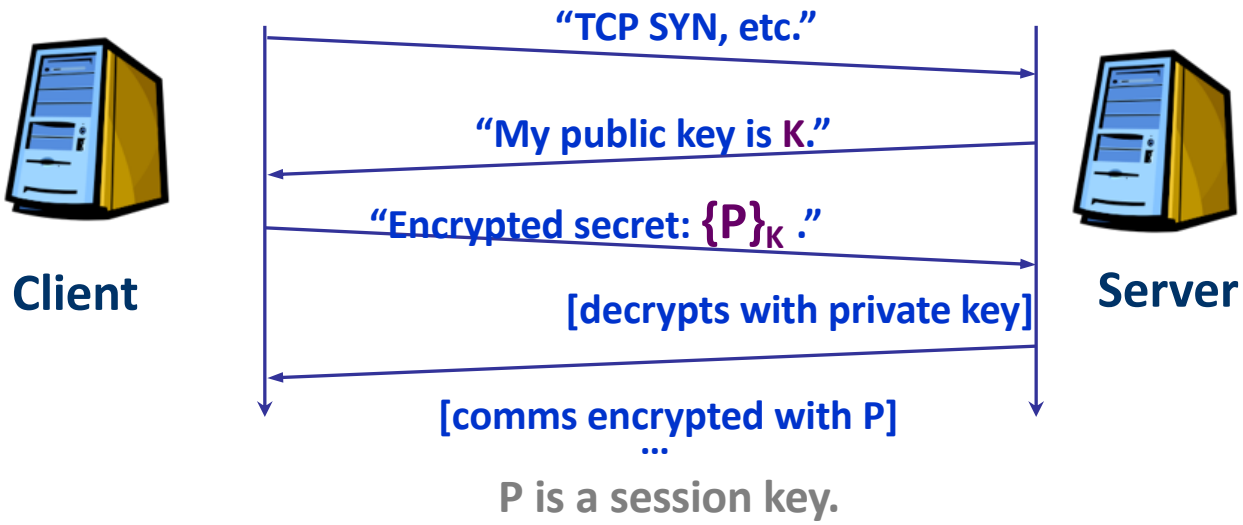
A and B can communicate securely in both directions if they each know the other's public key.



TLSv1.2

Cartoonishly Oversimplified, Version 1

- "Classic" Transport-Layer Security (e.g. TLS 1.2) is based on RSA.
- This approach is now deprecated due to lack of **forward secrecy**: compromise of the server's private key reveals all previous session keys.



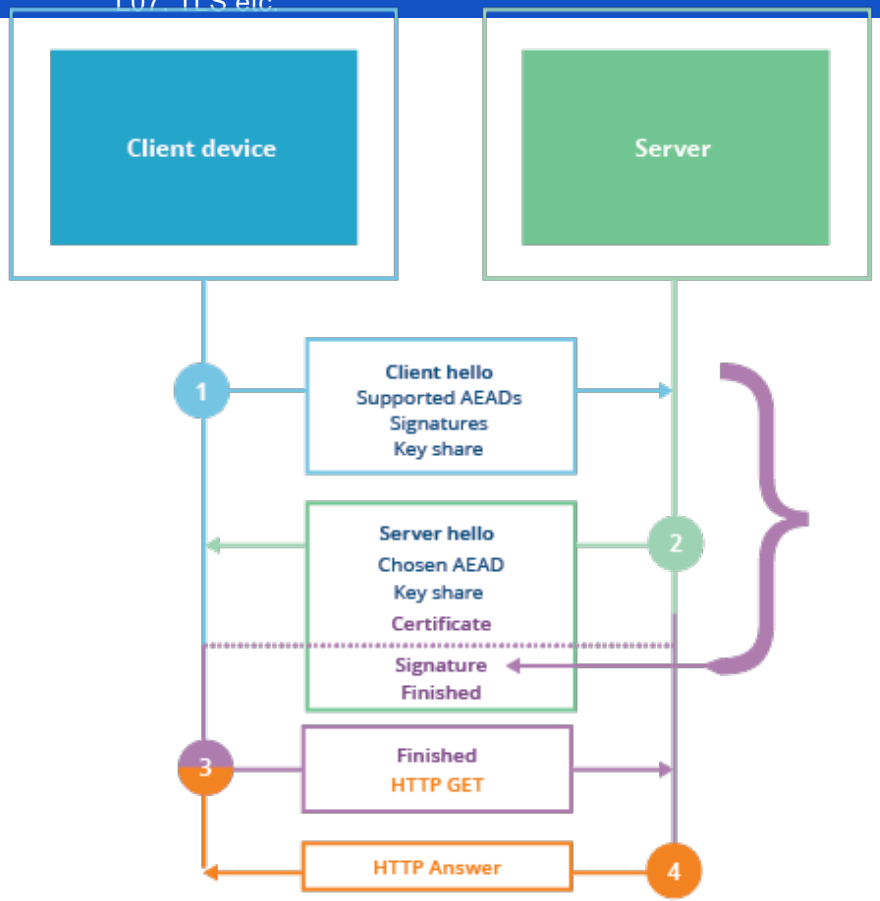
The problem of forward secrecy

- What if **M** hacks server **S** and steals its private key?
- Suppose further that **M** intercepts and records prior sessions of encrypted traffic to/from **S**.
  - **M** can "recover" all session keys (P) in the recorded sessions! (**M** decrypts P with private key of **S**.)
  - Therefore, **M** can decrypt any past session! (With its P.)
- TLS 1.2 lacks desired property of **forward secrecy**—an assurance that secret traffic stays secret in the future.
- TLS 1.3 uses Diffie-Hellman key exchange to provide it.

Why does the server provide a certificate if its public key is not used to exchange the session key?

- It is always necessary for the server to present a certificate so the client can **authenticate** the server.
  - E.g., in HTTPS, the lock icon says that the server is a valid server for the domain name in the URL.
- To authenticate the server, the client verifies that the server possesses the private key matching the public key in its certificate.
- Both versions of TLS protocol (classic 1.2 and 1.3) support this certificate-based authentication, but in different ways.
  - In TLS 1.2 the server proves it has the private key by using it to decrypt the session key and getting it right—Mal can't do that!
  - In TLS 1.3 the server **signs the handshake** with its private key, and the client verifies that the signature checks out.
- Understand how that protects the session key from Mal’s malice.

TLS 1.3



https://blog.cloudflare.com/rfc-8446-aka-tls-1-3/

CLOUDFLARE

End-to-end messaging: Signal protocols

- Scenario: a long-lived chat between Alice and Bob
- **Somebody's** device will be compromised **sometime**
- **Asynchronous**: the other party may be offline, even at connection establishment. ("Hiya! Let's chat!")
- Want **forward secrecy**
- And also **post-compromise security**: an ability to continue the session after an intruder has been evicted.
- **Double-ratchet protocol** widely adopted.

Signal

Signal: The Basics

- Encrypt-then-HMAC for every message.
- A new **ephemeral** session key for each message.
- Discard keys immediately!
- Provider does not know the keys. Only the participants.
- Outside the threat model (no defense):
  - User shares contents of chat with Eve (e.g., interrogation).
  - User retains copies of chat in cleartext for stealing.
  - User includes adversary in group chat by accident.
  - Secret compromise of endpoint (device or messaging app).

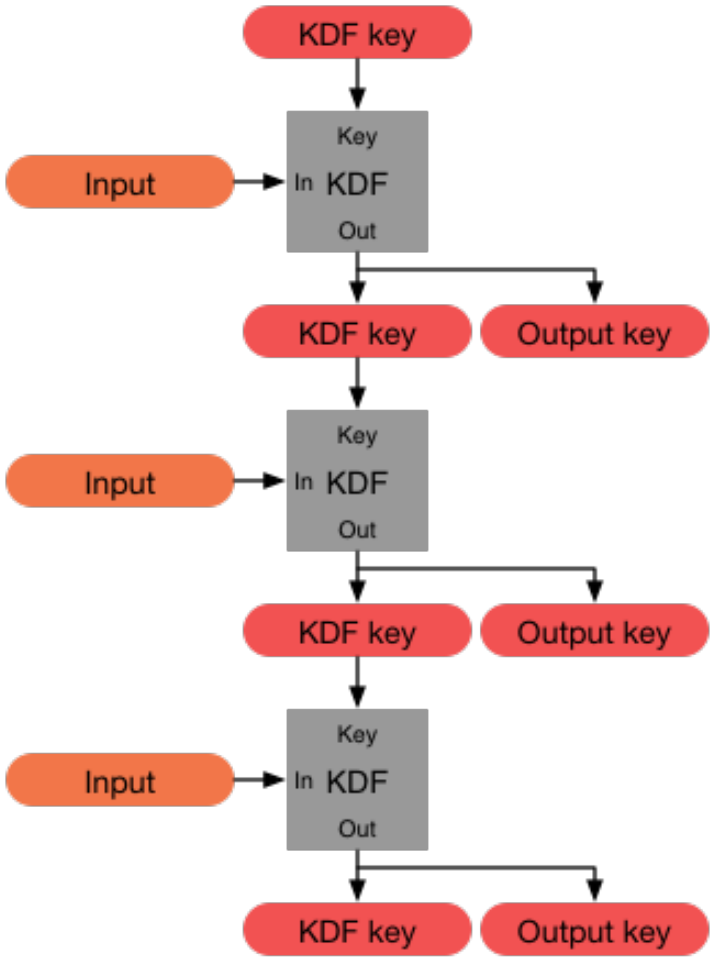
Signal details

- Session establishment makes a **root key**, a long-lived shared secret, seeded by multiple DH secrets.
- Uses ECDH to combine **multiple per-user keypairs** and an **ephemeral keypair** for the session, created by initiator.
  - The different keypairs have different terms and scope, for various compromise resistance. Public keys are downloaded from provider.
- Root key seeds a **chain** that generates the sequence of message keys. Actually two chains, one for each direction.
- Root key is also chained: both parties may pass new public DH material to seed a new root key with new DH secrets.

Chained keys

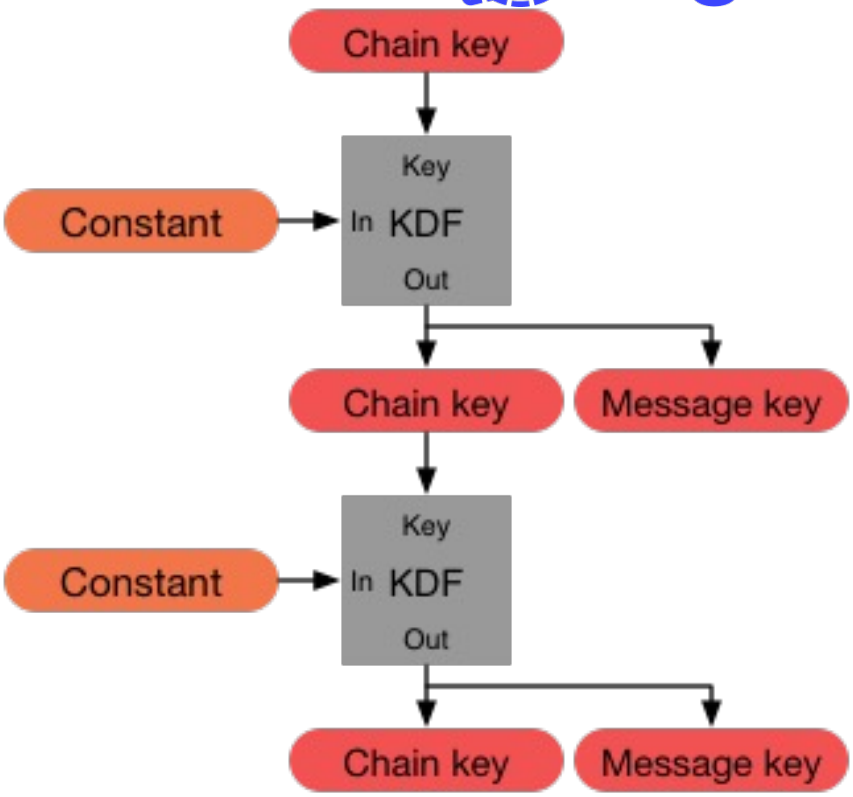
- Each chain generates a sequence of output keys in deterministic fashion.
  - All parties generate the same keys.
  - Root keys output from root chain seed new message key chains.
- Each chain is seeded and periodically freshened with shared secrets.
  - E.g., DH secrets from exchange of publics
- Key Derivation Function (KDF) is e.g., a variant of HMAC.

https://signal.org/docs/specifications/doubleratchet/



Signal's "Double Ratchet"

- Key Derivation Function (KDF) is a one-way **ratchet: "no going back".**
- Attacker cannot derive past keys from chain state (forward secrecy)
- Root chain is periodically refreshed with new DH secrets that the attacker does not know. (post-compromise security)
- Each new root key resets the message key chains.



Review: where are we?

- We can do secure communication between two endpoints
- With varying degrees of security against adversaries of various strengths, or more/less involved threat models.
- But: we still have not said how anyone knows for sure who they are talking to: **authentication**.
  - Option 1: secure channel. E.g., Alice and Bob meet for coffee or talk on a Zoom and share the QR codes for their keys.
  - Option 2: certified trust from a mutually trusted authority that endorses the public keys. Next time!

FIN